

Challenge Write-Up Template

Challenge Information

Student Name	Pascua, Diana KC S.
Section / Class	BSIT-3G
Challenge Name	SUDO MAKE ME A SANDWICH
Category	General Skills
Points / Difficulty	Easy
Date Solved	8/17/26
Flag Format Given	picoCTF{...}

1. Challenge Description

This challenge dropped me into a webshell on a remote box (green-hill.picoctf.net) as a low-privilege user, ctf-player. The hints — "What is sudo?" and "How do you know what permission you have?" — made it pretty clear the goal wasn't to find some hidden file through cleverness, but to figure out what my account was actually allowed to do as root, and then abuse that permission to get an honest root shell and read a flag that only root could reach.

2. Initial Analysis / Reconnaissance

Since the hints pointed straight at sudo and permissions, the first thing I checked was `sudo -l`, which tells you exactly which commands your account is allowed to run as another user (usually root) and whether a password is required. That's the standard first move any time a challenge mentions sudo — instead of guessing at exploits, you just ask the system what you're already allowed to do. The output showed that Emacs was on the NOPASSWD list, which immediately told me the category: this wasn't a crypto or web challenge, it was a classic Linux privilege-escalation challenge built around a program with dangerous sudo permissions — the same idea documented on sites like GTFOBins for editors, pagers, and interpreters that can spawn subshells.

3. Tools and Commands Used

Tool / Command	What it was used for	Result / Output
<code>sudo -l</code>	Check what commands my ctf-player account is allowed to run with sudo, and whether any of them need a password.	Listed emacs as a command I could run as root via sudo with NOPASSWD — meaning I could launch it as root without ever entering a password.
<code>sudo emacs</code>	Actually launch Emacs using the root privilege sudo -l said I had.	Emacs opened normally, but now the whole process — and anything spawned from inside it — was running as root instead of as ctf-player.
M-x shell (inside Emacs)	Use Emacs's built-in ability to open an interactive shell buffer, since Emacs is really just a very capable text editor that can spawn subprocesses.	A live root@challenge shell prompt appeared inside the Emacs buffer, since the child shell inherits the privileges of the Emacs process that spawned it.
<code>ls /root</code> <code>ls -la /root</code>	Check the obvious place first — root's home directory — for anything flag-related.	Only ordinary dotfiles (.bashrc, .emacs.d, .profile) — nothing named flag anywhere in /root.
<code>find / -name "flag.txt"</code> <code>2>/dev/null</code>	Search the whole filesystem for a file literally named flag.txt, since /root came up empty. Redirected stderr to /dev/null to hide all the "Permission denied" noise from directories I can't read.	Found exactly one match: /home/ctf-player/flag.txt.
<code>cat /home/ctf-player/flag.txt</code>	Print the contents of the file find located.	picoCTF{...}

4. Solution Walkthrough

Step 1

Find out what I was allowed to run as root

I ran `sudo -l` to list my sudo privileges. It showed I could run `/usr/bin/emacs` as root without a password. That's the whole vulnerability in one line: whoever set up this box gave a low-privilege user unrestricted, passwordless root access to a full text editor — and full-featured editors are never just editors.

Step 2

Launch the privileged program

I ran `sudo emacs`, which opened Emacs running as root. At this point I hadn't gained anything yet — I just had a root-owned Emacs process instead of a root shell.

Step 3

Break out into a shell

Emacs has a built-in command, M-x shell, that opens an interactive shell inside an editor buffer for running commands without leaving the editor. Because that shell is a child process of Emacs, and Emacs was running as root, the spawned shell inherited root's privileges too. That's the classic "GTFOBins" pattern: almost any program that can shell out — editors, pagers like `less/more`, even some interpreters — becomes a privilege-escalation tool the moment `sudo` lets you run it as root without restriction. My prompt changed to `root@challenge:/home/ctf-player#`, confirming I now had a genuine root shell.

Step 4

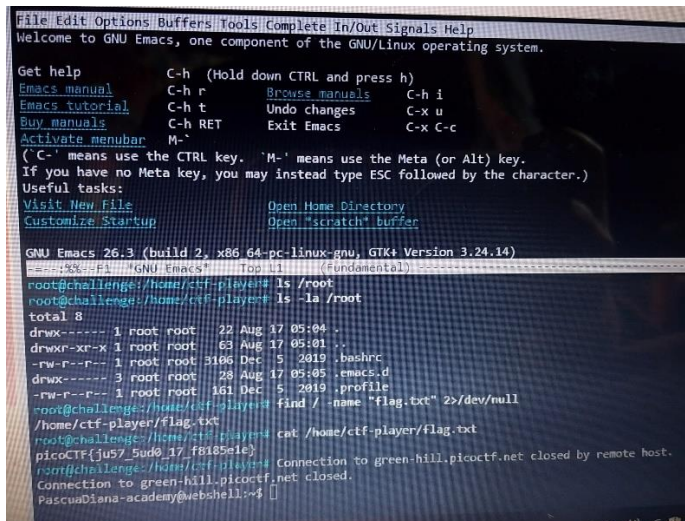
Look for the flag

I checked the obvious spot first with `ls /root` and `ls -la /root`, but that directory only had the standard dotfiles — no flag there. So I widened the search with `find / -name "flag.txt" 2>/dev/null` to hunt the entire filesystem for a file with that exact name, throwing away the permission-denied errors from directories I still couldn't read. That turned up `/home/ctf-player/flag.txt` — a file that lived in my own home directory the whole time, but almost certainly wasn't readable (or was a decoy) until I had root.

Step 5

Read the flag

`cat /home/ctf-player/flag.txt` printed `picoCTF{ju57_5ud0_17_f8185e1e}` directly to the terminal, which I then submitted on the challenge page for a confirmed "Correct flag!"

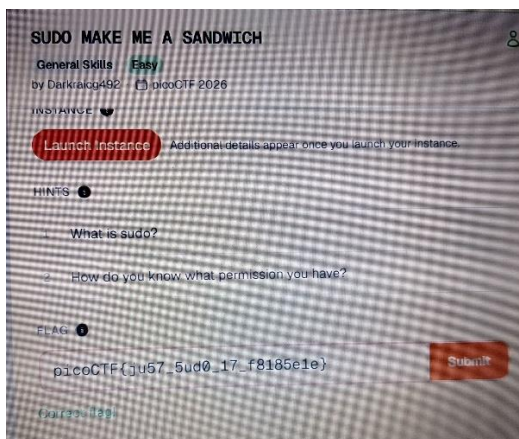


```
File Edit Options Buffers Tools Complete In/Out Signals Help
Welcome to GNU Emacs, one component of the GNU/Linux operating system.

Get help          C-h (Hold down CTRL and press h)
Emacs manual      C-h r          Browse manuals    C-h i
Emacs tutorial    C-h t          Undo changes      C-x u
Buy manuals       C-h RET       Exit Emacs        C-x C-c
Activate menubar  M-

('C-' means use the CTRL key. 'M-' means use the Meta (or Alt) key.
If you have no Meta key, you may instead type ESC followed by the character.)
Useful tasks:
Visit New File      Open Home Directory
Customize Startup   Open "scratch" buffer

GNU Emacs 26.3 (build 2, x86_64-pc-linux-gnu, GTK+ Version 3.24.14)
root@challenge:/home/ctf-player# ls /root
root@challenge:/home/ctf-player# ls -la /root
total 8
drwx----- 1 root root 22 Aug 17 05:04 .
drwxr-xr-x 1 root root 63 Aug 17 05:01 ..
-rw-r--r-- 1 root root 3106 Dec 5 2019 .bashrc
drwx----- 3 root root 28 Aug 17 05:05 .emacs.d
-rw-r--r-- 1 root root 161 Dec 5 2019 .profile
root@challenge:/home/ctf-player# find / -name "flag.txt" 2>/dev/null
/home/ctf-player/flag.txt
root@challenge:/home/ctf-player# cat /home/ctf-player/flag.txt
picoCTF{ju57_5ud0_17_f8185e1e}
root@challenge:/home/ctf-player#
Connection to green-hill.picoctf.net closed by remote host.
Connection to green-hill.picoctf.net closed.
PascuaDiana-academy@webshell:~$
```



5. Flag

picoCTF{ju57_5ud0_17_f8185e1e}

6. Key Concept / What I Learned

The core lesson is about the danger of granting NOPASSWD sudo access to interactive programs. `sudo` is meant to hand out narrow, specific privileges, but if the program it grants access to can itself spawn a shell or edit arbitrary files — like Emacs, Vim, less, or even some interpreters — then giving sudo rights to that program is functionally the same as giving full, unrestricted root access. This is exactly the class of misconfiguration cataloged by GTF0Bins, and the fix is always the

same: run `sudo -l` on any system you administer and treat every entry on that list as a potential root shell, not just a convenience.

7. References

GTFOBins (gtfobins.github.io) — a community-maintained catalog of Unix binaries that can be abused to bypass local security restrictions, including sudo misconfigurations for editors like Emacs. Consulted for background on why an editor's shell command becomes a privilege-escalation vector.
